```kotlin
fun main() {  new *
    for (i in 1 ≤ .. ≤ 10) {
        println("Hello, welcome to the tutorial")
    }
    TODO( reason: "Write some cool code so that they are impressed")
}
```

Programming Tutorial - 27

1. Tutorial - Objects & Classes

Kuzey Aydınhan | 7. November 2025

Contents

1. General Information

Tutorial
Structure
Time Plan
Questions

2. Java

3. Objects & Classes

4. Data Types

5. Conventions

About Me

Name: Kuzey Aydınhan
Age: 20
Study: 3rd Semester Bachelor of CS
Email Address: ujvea@student.kit.edu

It's my first time as a tutor → Feedback appreciated

About You

Introduction Round

- Name
- Study
- Semester
- Programming Knowledge

Tutorial

General

- Fridays 11:30-13:00 in Room -107 (50.34)
- Slides will be uploaded to the ILIAS-Page
- There is no compulsory attendance(Anwesenheitspflicht). You don't have to inform me or apologize to me
- You are allowed to visit other tutorials after speaking to the tutor

Tutorial

General

What will we do?

- Repeat lecture topics
- Solve some problems (Please bring a computer or pen & paper)
- Clear up your questions

Tutorial

General

What should we do?

- Repeat topics?
- Solve lots of problems?
- Bit of both?

Tutorial

Important

ILIAS

Don't forget to join the ILIAS-Page: https://ilias.studium.kit.edu/ilias.php

Tutorial

Important

Artemis

Login to Artemis: https://artemis.cs.kit.edu/

Structure

- In order to pass this module, you need **Übungsschein and the final projects(Abschlussaufgaben)**
- Übungsschein consists of the homeworks(5 in total) and the written exam(Präsenzübung)
- Final projects are 2 homework-like coding work. You will have circa 1 month for each

Info

If you get the Übungsschein this semester it means you won't have to get it again in the future. So even if you don't pass the final projects this semester you will have to only pass the projects in the future and don't repeat the homeworks again.

Structure

Important

Final Projects

You can't do the final projects without an Übungsschein. First you need to get the Übungsschein

O-Exam

Since programming is an Orientation-Exam(Orientierungsprüfung) it needs to be passed before the end of 3rd semester

About Homeworks

- You need >50% of total to pass (50P)
- Hand it over in Artemis
- Can't turn it in after the deadline (automatic system)

Tipps

- The homeworks become harder quickly → try to get many points in the first couple of them
- Plan more time for error and problem margins
- You can find guidelines in the Programming Wiki

About Präsenzübung

- You need >75% of total to pass
- No materials allowed in exam
- The solutions are to be written on paper
- You need to remember the tutorials number: **27**

About Final Projects

- 2 big coding projects
- For each you get 1 month
- Each project is of 20 points
- You need around 50% (>20) to pass

Tipps

- Start early as possible
- The first project usually collides with other exams. Don't avoid it and focus on other exams. This usually results in 1 week time for the first project → Not many points
- The second project is usually easier, try to get some points from the first one to not risk it

AI Usage & Plagiarism

- **Do not copy code from others.** Plagiarism will be detected by the system
- Try to use AI **minimally**!
- Usually results in confusion/bad code
- Can be used for debugging exceptions

Time Plan

Classes & Tutorials

- Until 21.02.
- No Classes/Tutorials between 24.12. - 06.01.

Exams

- Written Exam(Präsenzübung) on 21.01. between 17:30-20:00
- Final Project 1: Between 18.02. - 19.03.
- Final Project 2: Between 04.03. - 02.04.

Time Plan

Homeworks

Homeworks

- Homework 1: Between 05.11. - 20.11.
- Homework 2: Between 19.11. - 04.12.
- Homework 3: Between 03.12. - 18.12.
- Homework 4: Between 17.12. - 15.01.
- Homework 5: Between 14.01. - 29.01.

More information under: https://sdq.kastel.kit.edu/wiki/Vorlesung_Programmieren_WS_2025/26

Questions

Who to ask?

1. Check the FAQs
2. Check the Wiki
3. Check the ILIAS-Forum for a similar question
4. Ask the question in ILIAS-Forum (use a strong title)
5. Ask your tutor (check the contact info)
6. Write an email to: programmieren-vorlesung@cs.kit.edu

How to Reach Me

Contact Info

- Discord: `@sadspongebob_`
- E-Mail: ujvea@student.kit.edu (use your student email)
- Anonym: ILIAS-Tutorial-Forum
- & of course in person

Any Questions?

Java

What is Java?

- Object-Oriented Programming(OOP) language
- Platform independent
- Good for beginners

Java

How Java Works

1.Source Code

- Requires a text editor
- Write in readable Java language

2.Compile Source Code

- Command: `javac Filename.java`
- Translates Java source code (.java) to Bytecode(.class)
- Requires compiler(JDK)

3.Run Bytecode

- Command: `java Filename`

Recompile

After each change in code, it must be recompiled. The changes will not be automatically synchronised.

Java

JRE vs JDK

Java **R**untime **E**nvironment

- Required for running Bytecode

Java **D**evelopment **K**it

- Required for writing Java programms
- Includes JRE, Compiler and other Java tools

How to Install Java(JDK)

Artemis accepts Java 21, your code must be compatible with Java 21

- Oracle JDK: https://www.oracle.com/de/java/technologies/downloads/#java21
- Open-JDK: https://jdk.java.net/archive/

You can find more information under: Programming-WiKi

IDEs

- IDE: Integrated Development Environment
- They are better text editors to help write code
- Makes life easier!

IDEs

IntelliJ(Best of all)
https://www.jetbrains.com/idea/

Eclipse(Used in the classes)
https://eclipseide.org/

Visual Studio Code
https://code.visualstudio.com/Download

Java Basics

```java
class Car {
    String name;
    String color;

    void drive() {
        // Drive the car
        int velocity = 5;
    }

    /*
    Change
    the
    color
    of
    the
    car.
     */
    void changeColor(String color) {
        this.color = color;
    }
}

```

- Classes
- Attributes
- Methods
- Parameters
- Variables

Main Method

- Entry point of a program
- Max 1 `main` Method per program
- The program ends after `main`

```java
class Program {
    public static void main(String[] args) {
        // Run a program through here
    }
}

```

- `String[] args` passes in the commandline arguments

Main Method

Demo Program

- Compile & Run
- Input: 2 `Strings`
- Output: `"Say #1 to #2"`

What is OOP?

- Object-Oriented Programming
- Main idea: modelling objects, as close to the reality as possible
- Modelling $\neq$ Programming

Why OOP?

Pros of OOP

- Modularity
- Adaptibility
- ...
- Reusability
- Understandibility (especially by big projects)

Example

- Car → identity
- Which properties does it have? → state
- What can it do? → act

Example

in Java

- Class declaration with:
  `class ClassName`
- State is saved under **attributes**
- Act is defined through **methods**

```java
class Car {
    // Attributes
    String brand;
    String model;
    String color;
    int buildDate;
    Boolean isEngineOn;

    // Methods
    void turnOn() {
        // Turn the engine on
    }

    void drive() {
        // Drive the car
    }

    void turnOff() {
        // Turn the engine off
    }
}
```

Abstraction

```java
class Date {
    // Attributes
    int day;
    int month;
    int year;

    // Methods
    // e.g. addDate(), today()...
}

```

- Reusable
- Readable
- Better structured

Abstraction

```java
class Date {
    // Attributes
    int day;
    int month;
    int year;

    // Methods
    // e.g. addDate(), today()...
}

```

```java
class Car {
    // Attributes
    String brand;
    String model;
    String color;
    Boolean isEngineOn;

    // Usage of Date
    Date buildDate;

    // ...
}

```

Why abstract?

- Shorter, understandable class declarations
- Intuitive structure (Data Encapsulation)
- Reusability
- ...

Exercises

Exercise 1: Model the CS-Building

Exercise 2: Model chess

Guiding questions:

- Which "objects" are similar? → Classes
- What properties do they have? → Attributes
- What should they do? → Methods

Instantiation

```java
class Program {
    public static void main(String[] args) {
        // Create car
        Car car = new Car();

        // Set car attributes
        car.brand = "mercedes";
        car.color = "black";
        car.buildDate = new Date(13, 6, 2025);
        car.model = "c200";

        // Drive car
        car.turnOn();
        car.drive();

        // Create another car
        Car secondCar = new Car();
        secondCar.color = "grey";

        //...
    }
}
```

- The **instance** of a class is an **object**
- Create new instances with `new Classname()`
- Get/Set attributes with `object.attribute`
- Use methods with `object.method()`

Data Types

- Data types are a set of similar values
- Types set the possible values for variables
- Types set which operations can be used on which variables

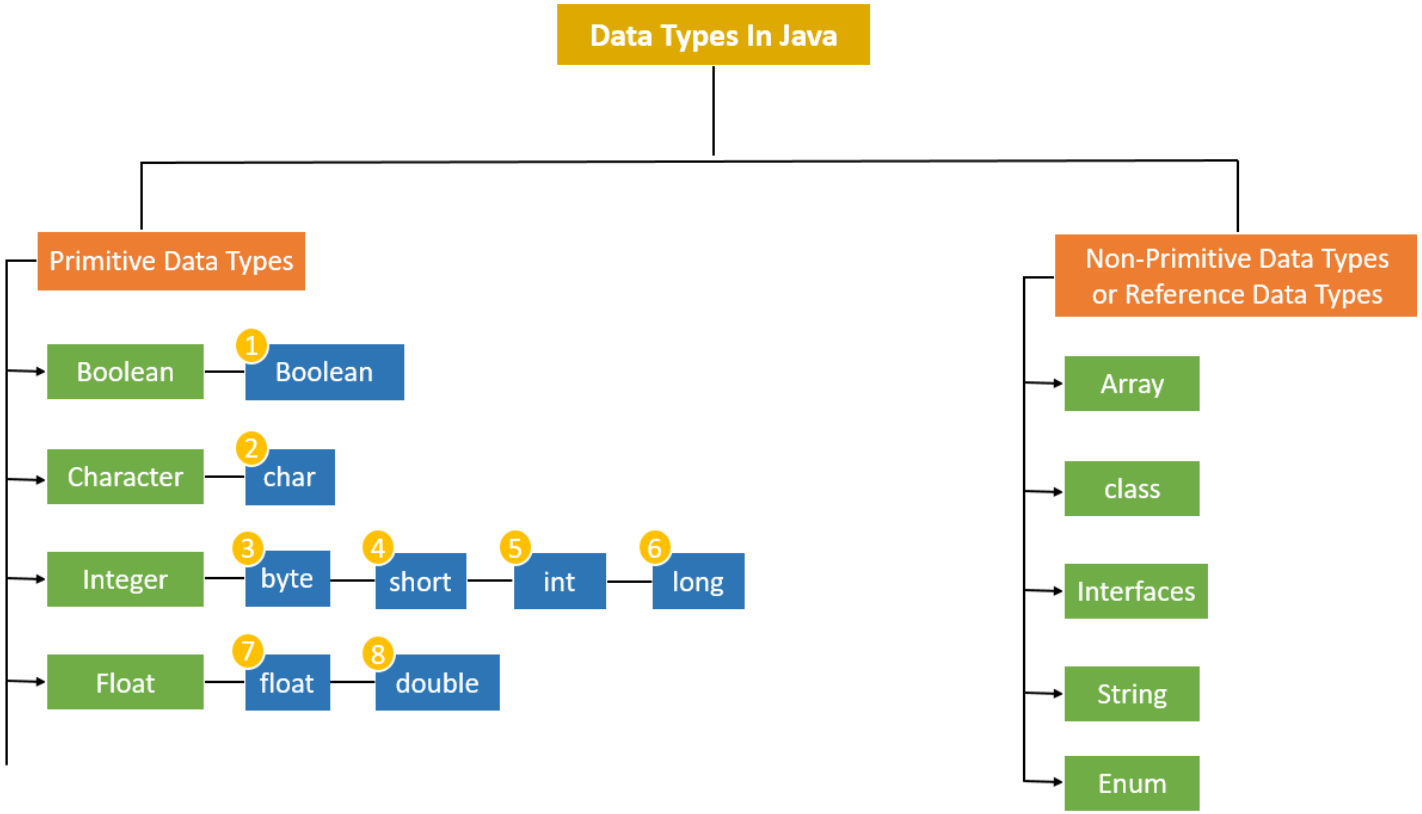
In Java:

- 8 elementary (primitive) data types
- Extendible through classes and enums

Primitive Data Types

Data Type	Value Type	Space	Example
`boolean`	truth	8 bit	`true, false`
`int`	integer	32 bit	`0,1, 2025`
`long`	big integer	64 bit	`12l`
`float`	floating-point	32 bit	`0.234f, 2f`
`double`	big float	64 bit	`0.379E-8`
`char`	unicode	16 bit	`'A', '\u05D0'`
`byte`	integer	8 bit	`-25`
`short`	integer	16 bit	`16`

Non-Primitive Data Types



Enums

What are enums?

- Special classes, that define specific constants

Why enums?

- Better readability
- Reduces error sources by using predefined constants
- Additional functionality

```
enum CompassDirection { NORTH, EAST, SOUTH, WEST };

enum Color {
    RED,
    GREEN,
    BLUE;
};

```

Naming Conventions

Naming conventions are optional but strongly recommended rules. Basically, how to name the **classes, methods, attributes etc.** Helps with:

- Consistency
- Readability
- Prevention of misunderstandings

Example naming conventions:

- camelCase
- PascalCase
- ALL_CAPS
- ...

Naming Conventions

in Java

Element	Convention	Example
Classes	PascalCase	`class CarProgram`
Variables	camelCase	`String color = "yellow"`
Methods	camelCase	`void drive()`
Enum Values	ALL_CAPS	`NORTH_WEST, RED`

Other General Conventions

- English coding: not using Ä, Ö..
- Java file names should match the class inside:
  `class Car` → Car.java

Conclusion

- Sourcecode → Compile → Run Bytecode
- Java programs start and end with `main` method
- OOP is an approach for modelling softwares
- Classes are the blueprints of objects
- Objects are instances of classes
- Strings and enums aren't primitive
- Enums help write clean code
- Naming conventions help write consistent code

Remember

Important

Join the ILIAS-Page
Login to Artemis

Demo Program

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        if (args.length != 2) {
            return;
        }
        String first = args[0];
        String second = args[1];
        System.out.println("Say " + first + " to " + second);
    }
}

```

```
PS > javac Main.java
PS > java Main.java hi me
Say hi to me
```

Solution

Exercise 1

```java
class Building {
    String name;
    int capacity;
    Room[] rooms;

    int openingTime;
    int closingTime;
    boolean isOpen;

    boolean isBuildingOpen(int currentTime) {
        // Check if currentTime is inside of working hours
        return true;
    }

    void closeBuilding() {
        // Close the building if necessary
    }
}

```

```java
class Room {
    int number;
    int floorNumber;

    boolean isOccupied;
    String occupiedBy;

    boolean occupyRoom(String username) {
        // Set the room to occupied if not already so
        return true;
    }
}

```

Solution

Exercise 2

```java
class Game {
    Board board;
    String player1;
    String player2;
    boolean isGameOver;

    void playGame() {
        // Runs the game in a loop
    }

    void setGameOver() {
        // Ends the game
    }
}

```

```java
class Board {
    String[][] board;

    void makeMove(Piece piece, int tileRow, int tileCol) {
        // Move the piece to the given tile
    }
}

```

Solution

Exercise 2

```java
class Piece {
    String name;
    int tileX;
    int tileY;
    boolean isCaptured;

    void move(int tileX, int tileY) {
        // Move to given tile
    }
}

```